CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 11: CNN for Image Classification
Problem Statement: A recycling plant wants a camera system that sorts waste on a conveyor belt into categories automatically. A garbage-classification image dataset (cardboard, glass, metal, paper, plastic, trash), or CIFAR-10 as a substitute. The objective is to build and train a CNN for multi-class image classification. Use TensorFlow, Keras, NumPy, Matplotlib.
Tasks:
1. Load, resize and normalise the images; split into train, validation and test sets.
2. Build a CNN with convolution, pooling, flatten and dense layers.
3. Train with categorical cross-entropy and plot accuracy and loss curves.
4. Show the confusion matrix and sample misclassifications.
5. Visualise the feature maps of the first convolution layer.

FORMULA:

1. Image Normalisation

$X_normalized = X / 255.0$

2. Convolution Operation

$Output(i, j) = \Sigma\Sigma\ X(i + m, j + n) \times K(m, n) + b$

3. ReLU Activation Function

$f(x) = max(0, x)$

4. Max Pooling

$P(i, j) = max\ \{ X(i + m, j + n) \}$

5. Softmax Activation Function

$P_i = e^{zi} / \Sigma\square\ e^{zj}$

6. Categorical Cross-Entropy Loss

$L = -\Sigma_i\ y_i\ log(P_i)$

7. Classification Accuracy

Accuracy = (Correct Predictions / Total Predictions) × 100%

8. Confusion Matrix

CM_{i}□ = Number of samples whose actual class is i and predicted class is j

9. Precision

Precision = TP / (TP + FP)

10. Recall

Recall = TP / (TP + FN)

11. F1-Score

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# 1. Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

classes = ["Airplane", "Automobile", "Bird", "Cat", "Deer",
           "Dog", "Frog", "Horse", "Ship", "Truck"]

# Normalise images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

# Split training data into training and validation sets
x_val, y_val = x_train[45000:], y_train[45000:]
```

```
x_train, y_train = x_train[:45000], y_train[:45000]

# 2. Build CNN
model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation="relu", name="conv1"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# 3. Train CNN
history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=10,
    batch_size=64,
    verbose=1
)

# Plot accuracy and loss
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("CNN Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
```

```
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training")
plt.plot(history.history["val_loss"], label="Validation")
plt.title("CNN Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

# Test evaluation
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
print("Test Loss:", round(test_loss, 4))

# 4. Confusion matrix
predictions = model.predict(x_test, verbose=0)
y_pred = np.argmax(predictions, axis=1)
y_true = np.argmax(y_test, axis=1)

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=classes)
disp.plot(xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title("CNN Confusion Matrix")
plt.tight_layout()
plt.show()

# Show sample misclassified images
wrong = np.where(y_pred != y_true)[0]
sample = wrong[:10]

plt.figure(figsize=(12, 5))
for i, idx in enumerate(sample):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[idx])
    plt.title(f"True: {classes[y_true[idx]]}\nPred: {classes[y_pred[idx]]}")
    plt.axis("off")
```

```
plt.suptitle("Sample Misclassified Images")
plt.tight_layout()
plt.show()

# 5. Visualise first convolution layer feature maps
first_conv = model.get_layer("conv1")
feature_model = tf.keras.Model(
    inputs=model.inputs,
    outputs=first_conv.output
)

feature_maps = feature_model.predict(x_test[:1], verbose=0)

plt.figure(figsize=(12, 6))
for i in range(32):
    plt.subplot(4, 8, i + 1)
    plt.imshow(feature_maps[0, :, :, i], cmap="viridis")
    plt.axis("off")

plt.suptitle("First Convolution Layer Feature Maps")
plt.tight_layout()
plt.show()
```

OUTPUT:

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ **2734s** 16us/step

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0

```
├─────────────────────────────────┼────────────────────────┼──────────────
─┤
│ conv2d (Conv2D)                 │ (None, 13, 13, 64)     │        18,496
│
├─────────────────────────────────┼────────────────────────┼──────────────
─┤
│ max_pooling2d_1 (MaxPooling2D)  │ (None, 6, 6, 64)       │             0
│
├─────────────────────────────────┼────────────────────────┼──────────────
─┤
│ flatten (Flatten)               │ (None, 2304)           │             0
│
├─────────────────────────────────┼────────────────────────┼──────────────
─┤
│ dense_3 (Dense)                 │ (None, 64)             │       147,520
│
├─────────────────────────────────┼────────────────────────┼──────────────
─┤
│ dense_4 (Dense)                 │ (None, 10)             │           650
│
└─────────────────────────────────┴────────────────────────┴──────────────
─┘
```

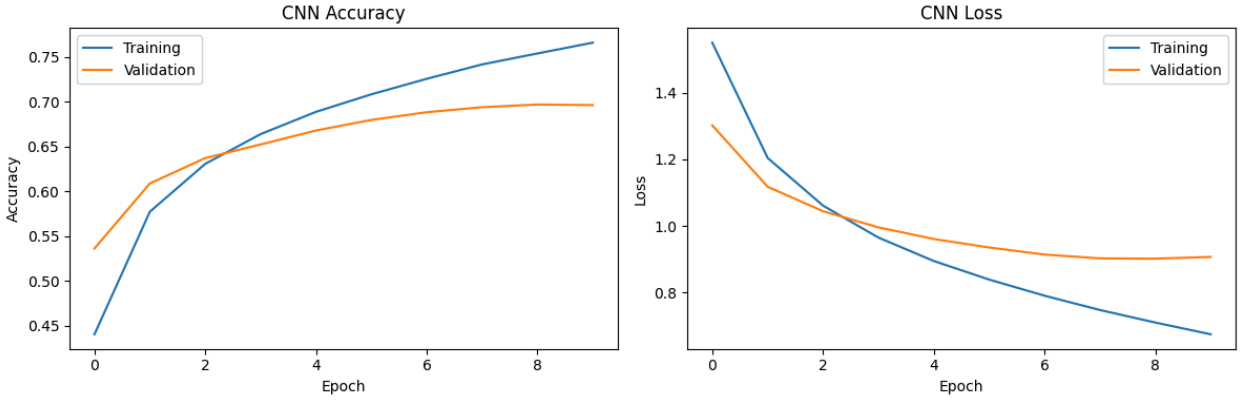
Total params: 167,562 (654.54 KB)

Trainable params: 167,562 (654.54 KB)

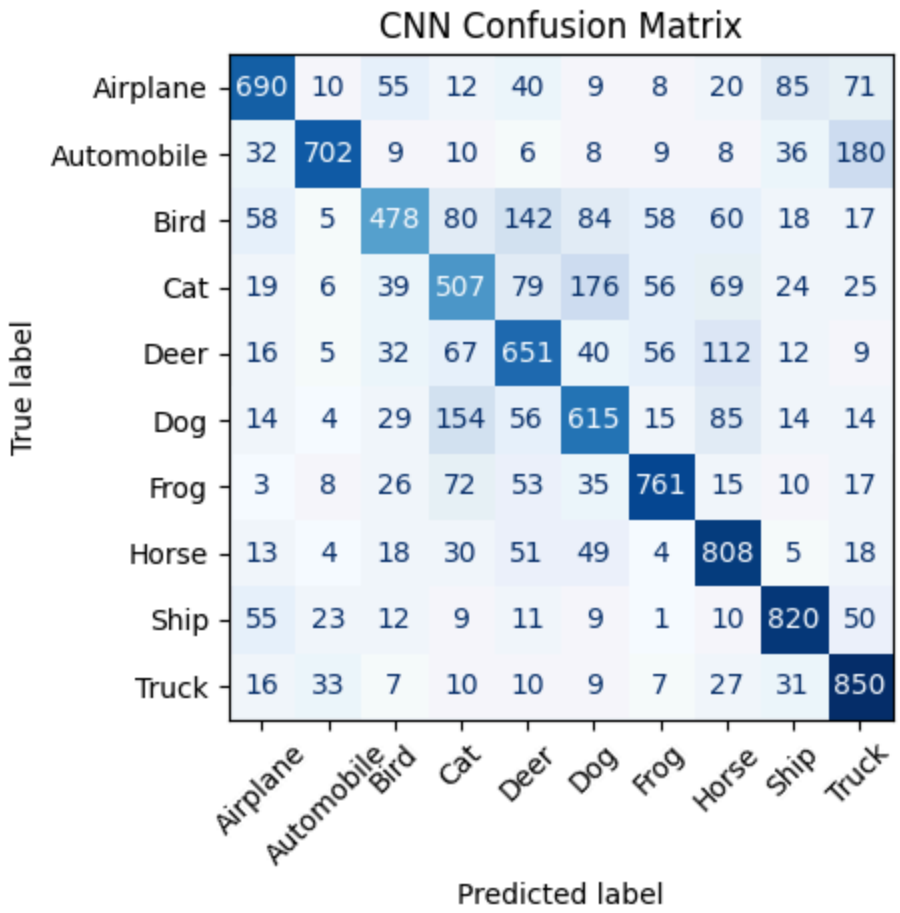
Non-trainable params: 0 (0.00 B)

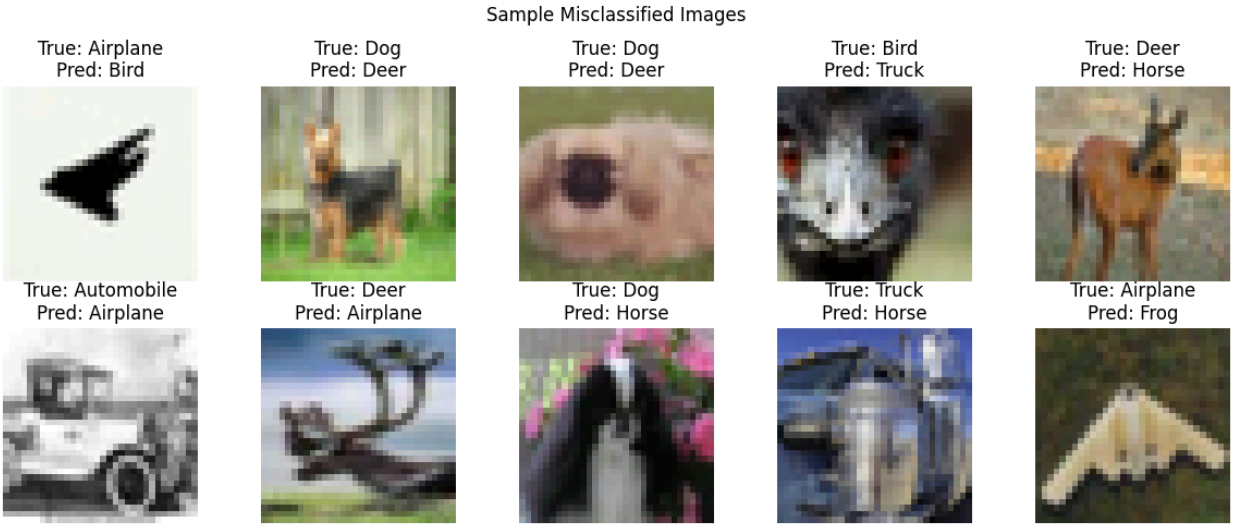
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **49s** 68ms/step - accuracy: 0.4404 - loss: 1.5497 - val_accuracy: 0.5362 - val_loss: 1.3016
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **47s** 67ms/step - accuracy: 0.5773 - loss: 1.2039 - val_accuracy: 0.6088 - val_loss: 1.1175
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **49s** 69ms/step - accuracy: 0.6307 - loss: 1.0609 - val_accuracy: 0.6372 - val_loss: 1.0440
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **81s** 68ms/step - accuracy: 0.6640 - loss: 0.9650 - val_accuracy: 0.6524 - val_loss: 0.9956
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **81s** 67ms/step - accuracy: 0.6888 - loss: 0.8947 - val_accuracy: 0.6680 - val_loss: 0.9607
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **48s** 68ms/step - accuracy: 0.7084 - loss: 0.8390 - val_accuracy: 0.6798 - val_loss: 0.9352

Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **48s** 69ms/step - accuracy: 0.7257 - loss: 0.7906 - val_accuracy: 0.6884 - val_loss: 0.9140
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **48s** 68ms/step - accuracy: 0.7418 - loss: 0.7478 - val_accuracy: 0.6940 - val_loss: 0.9026
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **82s** 68ms/step - accuracy: 0.7540 - loss: 0.7101 - val_accuracy: 0.6970 - val_loss: 0.9018
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ **48s** 68ms/step - accuracy: 0.7661 - loss: 0.6749 - val_accuracy: 0.6964 - val_loss: 0.9069



Test Accuracy: 68.82 %
Test Loss: 0.9401


CNN Confusion Matrix
Airplane 690 10 55 12 40 9 8 20 85 71
Automobile 32 702 9 10 6 8 9 8 36 180
Bird 58 5 478 80 142 84 58 60 18 17
Cat 19 6 39 507 79 176 56 69 24 25
Deer 16 5 32 67 651 40 56 112 12 9
Dog 14 4 29 154 56 615 15 85 14 14
Frog 3 8 26 72 53 35 761 15 10 17
Horse 13 4 18 30 51 49 4 808 5 18
Ship 55 23 12 9 11 9 1 10 820 50
Truck 16 33 7 10 10 9 7 27 31 850
Airplane
Automobile
Bird
Cat
Deer
Dog
Frog
Horse
Ship
Truck
True label
Predicted label


Sample Misclassified Images
True: Airplane
Pred: Bird
True: Dog
Pred: Deer
True: Dog
Pred: Deer
True: Bird
Pred: Truck
True: Deer
Pred: Horse
True: Automobile
Pred: Airplane
True: Deer
Pred: Airplane
True: Dog
Pred: Horse
True: Truck
Pred: Horse
True: Airplane
Pred: Frog

First Convolution Layer Feature Maps

